

CHAPTER – XI

EXCEPTION AND FILE HANDLING

EXCEPTION:

- An exception is an unexpected event that happens during program execution (runtime) that disrupts the normal flow of instructions.

TYPES OF EXCEPTION:

- Checked Exceptions (Compile-time Exceptions)
- Unchecked Exceptions (Runtime Exceptions)
- Errors

CHECKED EXCEPTIONS:

- These are exceptions checked and enforced by the compiler. You must handle them using try-catch or throws.

EXAMPLES:

- IOException
- SQLException
- ClassNotFoundException
- FileNotFoundException

UNCHECKED EXCEPTIONS:

- These occur during runtime and are not checked by the compiler.

EXAMPLES:

- ArithmeticException

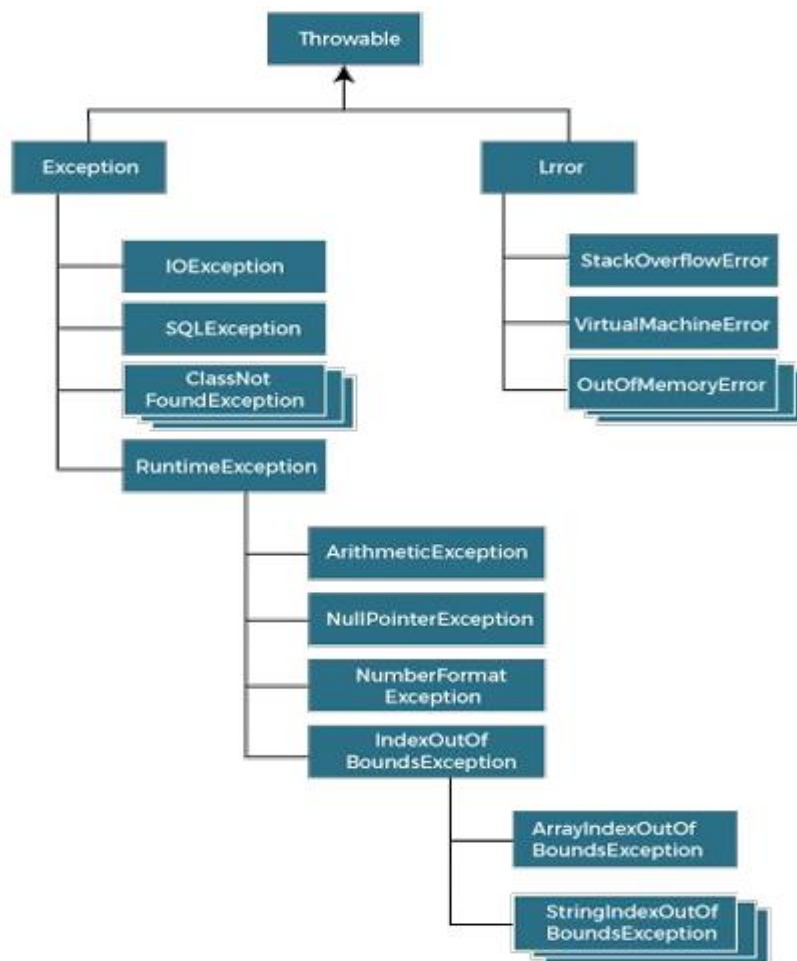
- NullPointerException
- ArrayIndexOutOfBoundsException
- NumberFormatException

ERRORS:

- Errors are serious issues that are not meant to be handled by application code. They occur due to system-level problems.

EXAMPLES:

- OutOfMemoryError
- StackOverflowError
- VirtualMachineError



EXCEPTION HANDLING METHODS IN JAVA:

- **try** – Wraps risky code
- **catch** – Handles the exception
- **finally** – Always executes
- **throw** – Manually throws exception
- **throws** – Declare the exception for the caller to handle

try – catch:

- The code that may cause an exception is placed inside a try block, and the exception is handled inside a catch block.

SYNTAX:

```
try{  
    //code may cause exception  
} catch (Exception e) {  
    //code to handle exception  
}
```

MULTIPLE CATCH BLOCKS:

- Can catch different exception types separately.

EXAMPLE:

```
public class MultipleCatch {  
    public static void main(String[] args) {  
        try {  
            int a = 10 / 0;  
        }  
        catch (ArithmeticException e) {
```

```
        System.out.println("Cannot divide by zero.");
    }
    catch (Exception e) {
        System.out.println("Some error occurred.");
    }
    System.out.println("Program ended.");
}
}
```

USING finally BLOCK:

- The finally block always runs—whether an exception occurs or not. It is commonly used for closing resources (files, database connections, etc.).

EXAMPLE:

```
public class FinallyExample {
    public static void main(String[] args) {
        try {
            int a = 10 / 0; // Exception
        }
        catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero.");
        }
        finally {
            System.out.println("Finally block is executed.");
        }
    }
}
```

```
    }  
  }  
}
```

USING throws KEYWORD:

- **throws** is used in the method signature to say: "This method might cause an exception; someone else must handle it."

EXAMPLE:

```
public class ThrowsExample {  
    static void divide() throws ArithmeticException {  
        int result = 10 / 0;  
        System.out.println(result);  
    }  
    public static void main(String[] args) {  
        try {  
            divide();  
        } catch (ArithmeticException e) {  
            System.out.println("Cannot divide by zero.");  
        }  
    }  
}
```

USING throw KEYWORD:

- **throw** is used inside a method to manually throw an exception.

EXAMPLE:

```
public class ThrowExample {  
    public static void main(String[] args) {  
        int age = 15;  
        if (age < 18) {  
            throw new ArithmeticException("Not eligible to vote");  
        }  
        System.out.println("Eligible to vote");  
    }  
}
```

CUSTOM EXCEPTION(USER – DEFINED):

- A Custom Exception (User-defined Exception) in Java is an exception that you create yourself to represent application-specific errors.

EXAMPLE:

```
class MyException extends Exception {  
    MyException(String message) {  
        super(message);  
    }  
}  
  
public class CustomExceptionExample {  
    public static void main(String[] args) {
```

```
int age = 15;
try {
    if (age < 18) {
        throw new MyException("Not eligible to vote");
    }
    System.out.println("Eligible to vote");
} catch (MyException e) {
    System.out.println(e.getMessage());
}
}
```